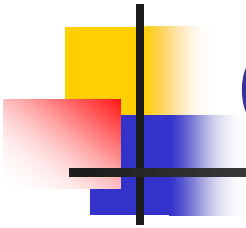
Projeto e Análise de Algoritmos

Grafos

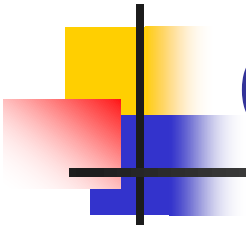
Universidade Federal do Amazonas
Instituto de Computação





Caminhamento em grafos

- Algoritmos de caminhamento determinam a ordem com que os vértices de um grafo serão visitados
- Pode ser feito em largura ou em profundidade



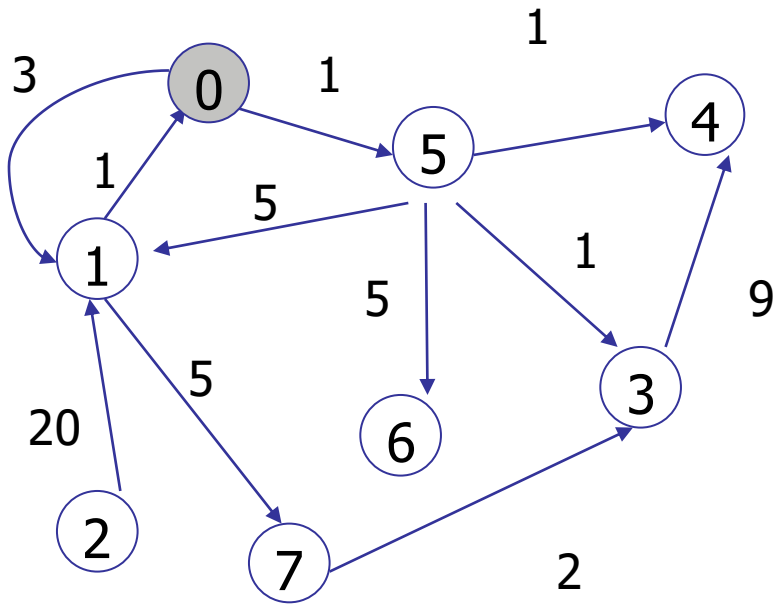
Caminhamento em Largura

- Caminha(v)
- Marcar v como visitado
- Visitar todos os adjacentes, guardando seus filhos em uma fila
- Para cada elemento da fila, marcar elemento como visitado e guardar todos os seus adjacentes em uma fila
- Proseguir operação até que não haja mais elementos a visitar

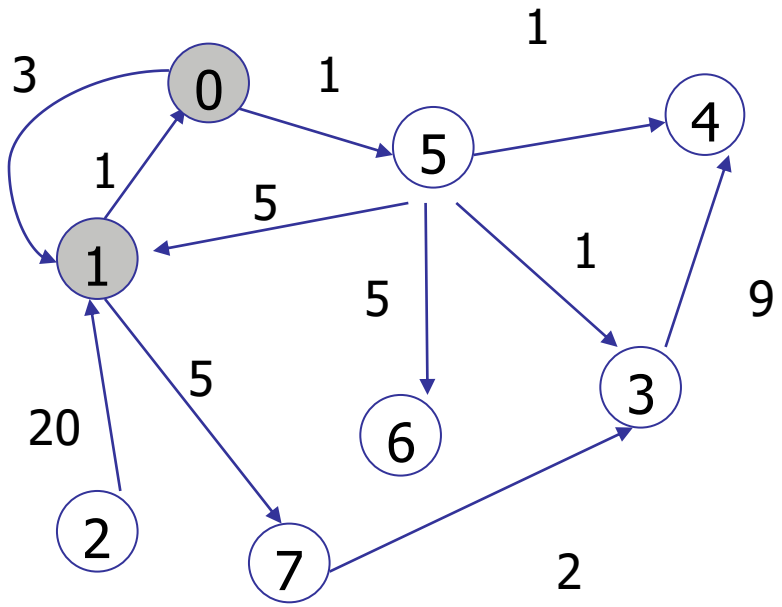
Caminhamento em Largura

- Gera uma árvore de caminhamento em largura

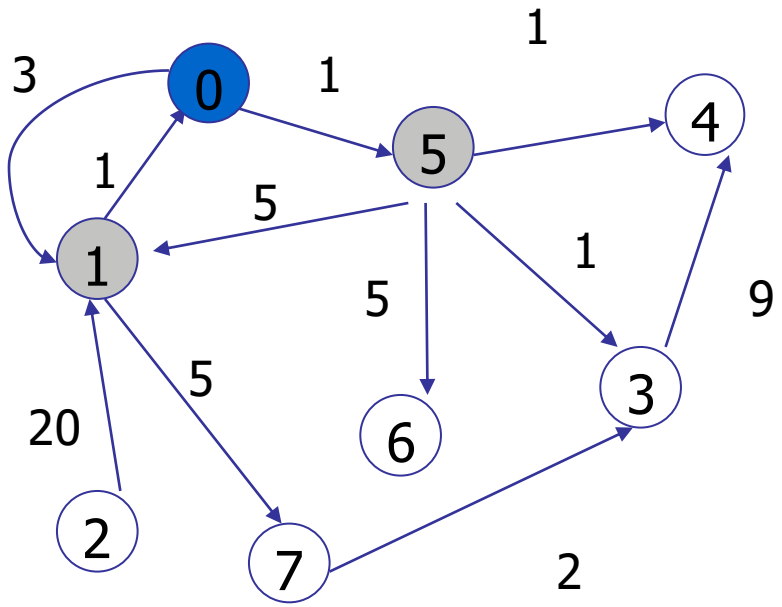
Caminhamento em largura



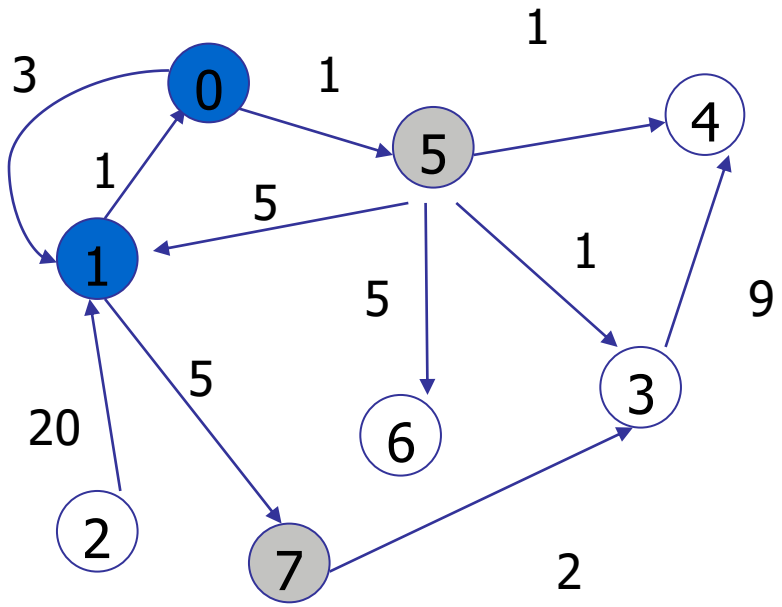
Caminhamento em largura



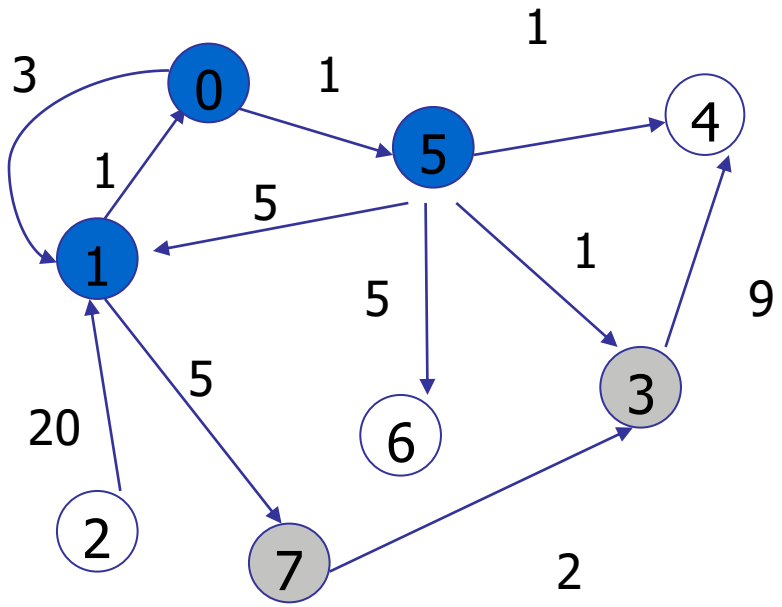
Caminhamento em largura



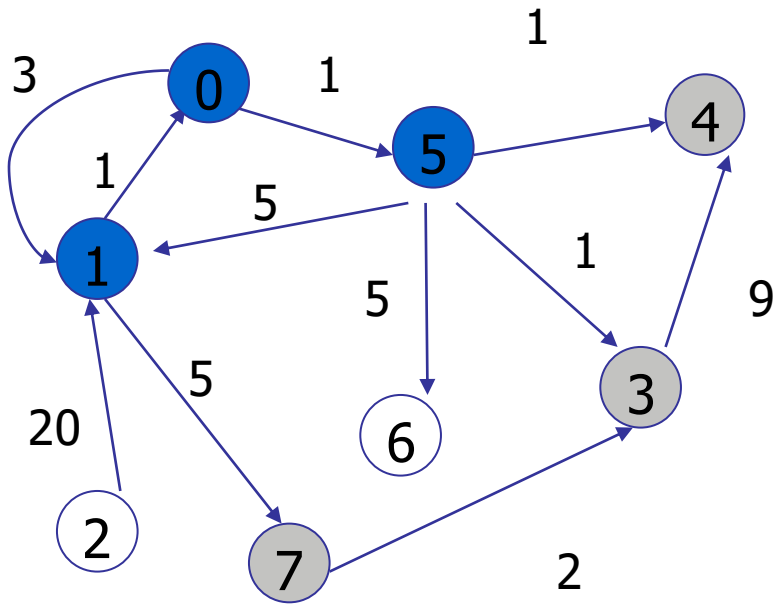
Caminhamento em largura



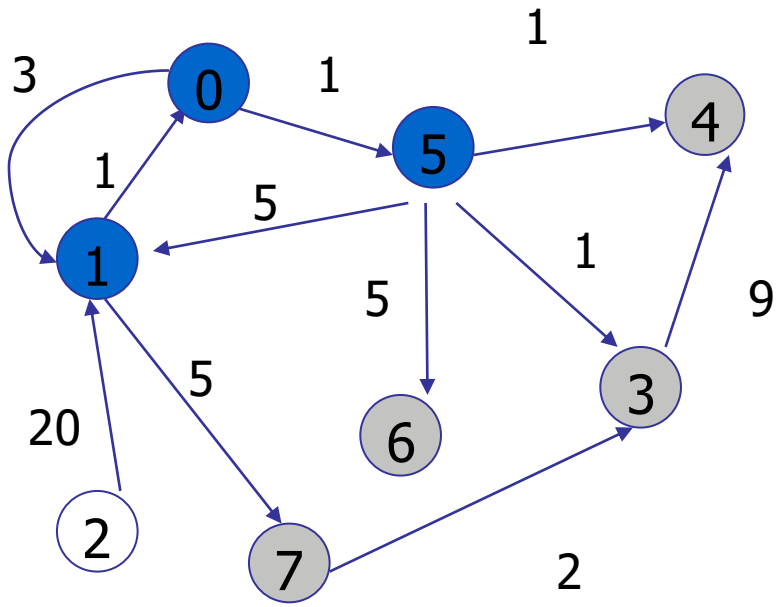
Caminhamento em largura



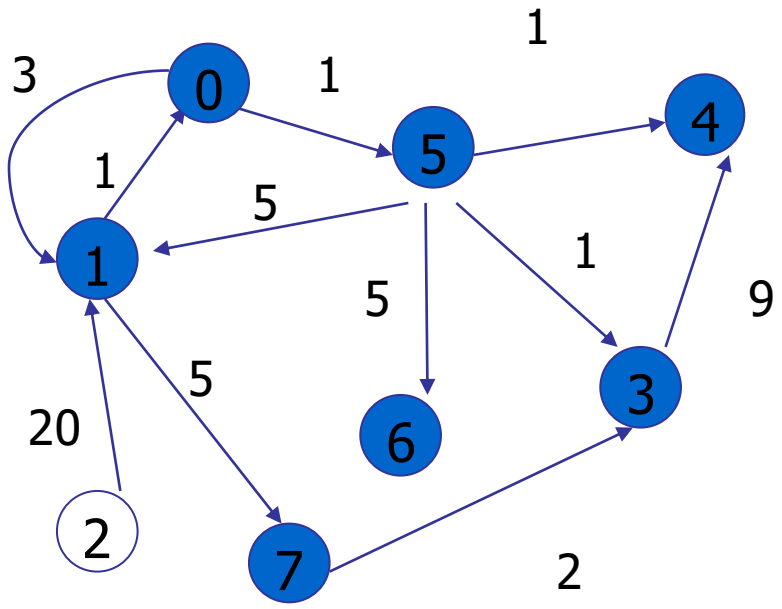
Caminhamento em largura



Caminhamento em largura

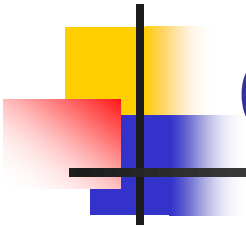


Caminhamento em largura



Caminhamento em Largura

- Calcula a distância (menor número de arestas) entre um ponto de partida e cada um dos vértices de um grafo



Caminhamento em profundidade

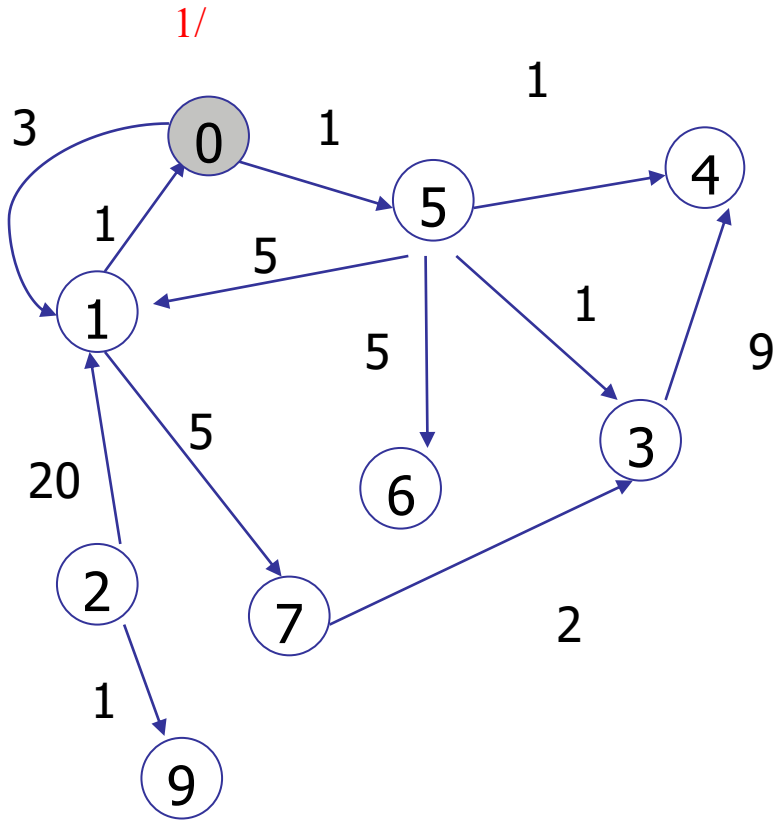
- Nós que são visitados pela primeira vez são marcados em cinza. O momento da primeira visita é definido como tempo de descoberta do nó. A cor cinza em v indica que ainda há vértices adjacentes a v que não foram visitados
- Quando não houver mais adjacentes a visitar a partir de um nó, ele é marcado em preto (azul nos exemplos). Este momento é denominado de tempo de termino

Caminhamento em Profundidade

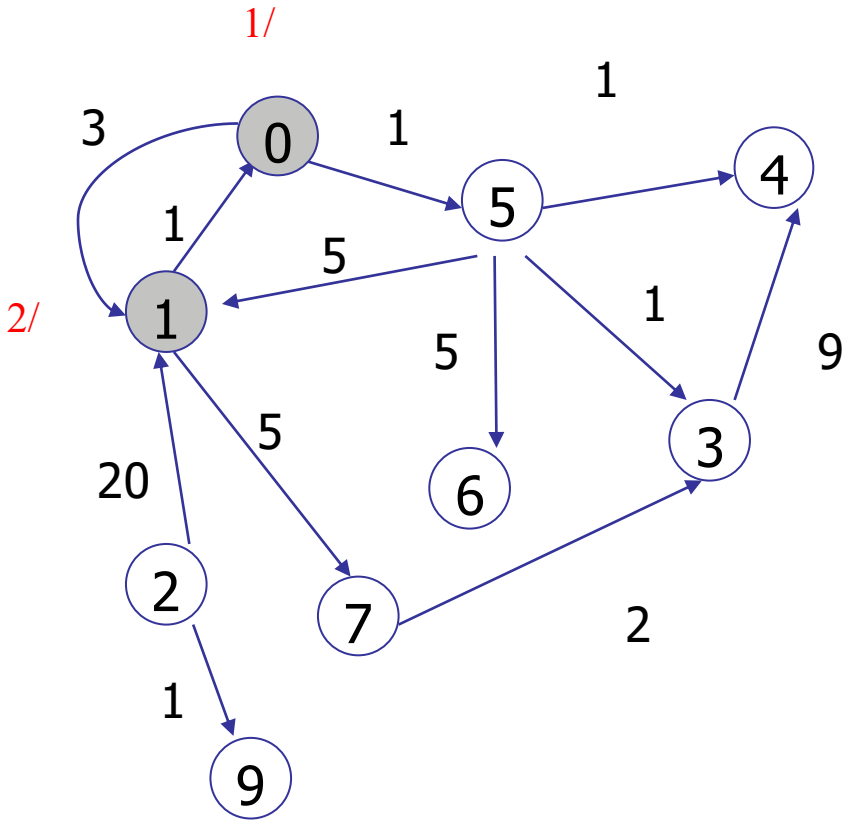
- Caminha(v)
- Marcar v como visitado
- Para cada adjacente(va) executamos caminha(va)

- Caminha é executado novamente para todo v que não tenha sido alcançado

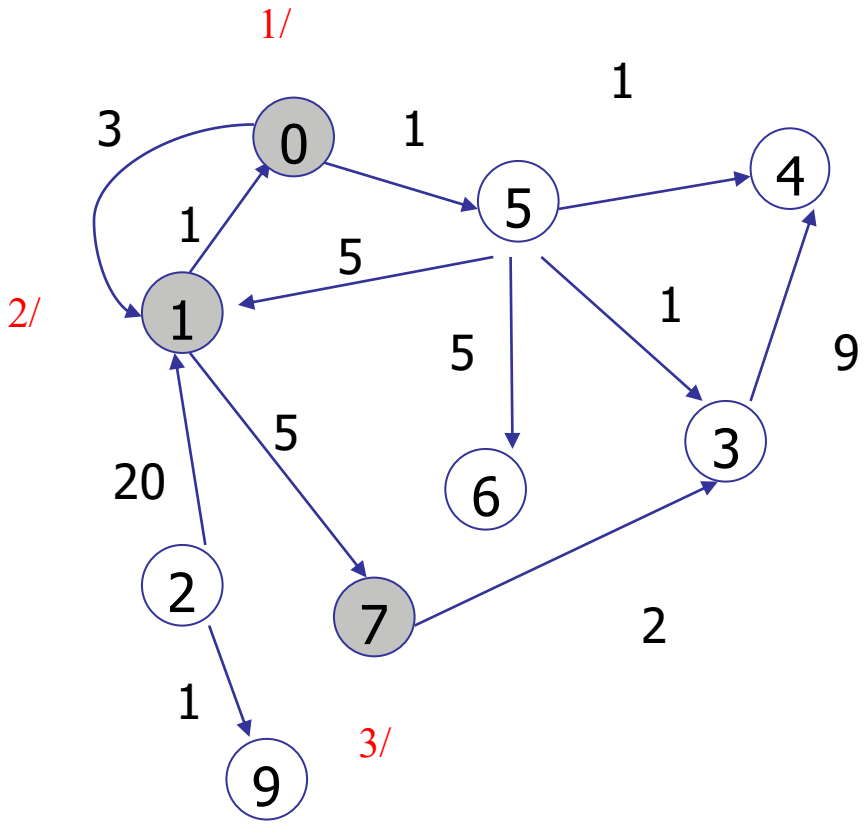
Caminhamento em profundidade



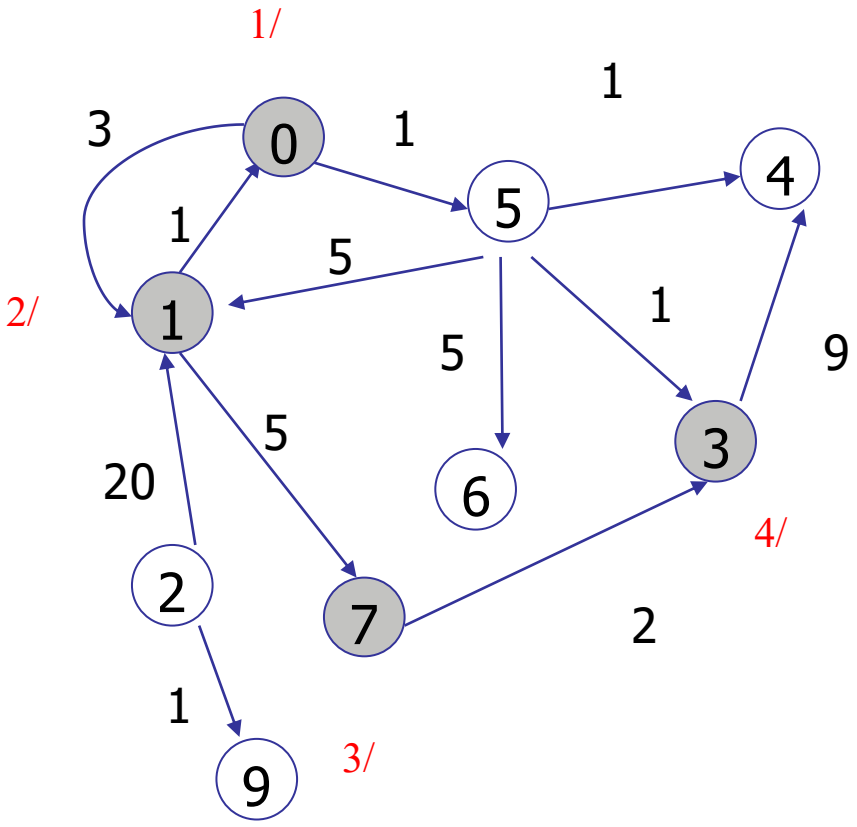
Caminhamento em profundidade



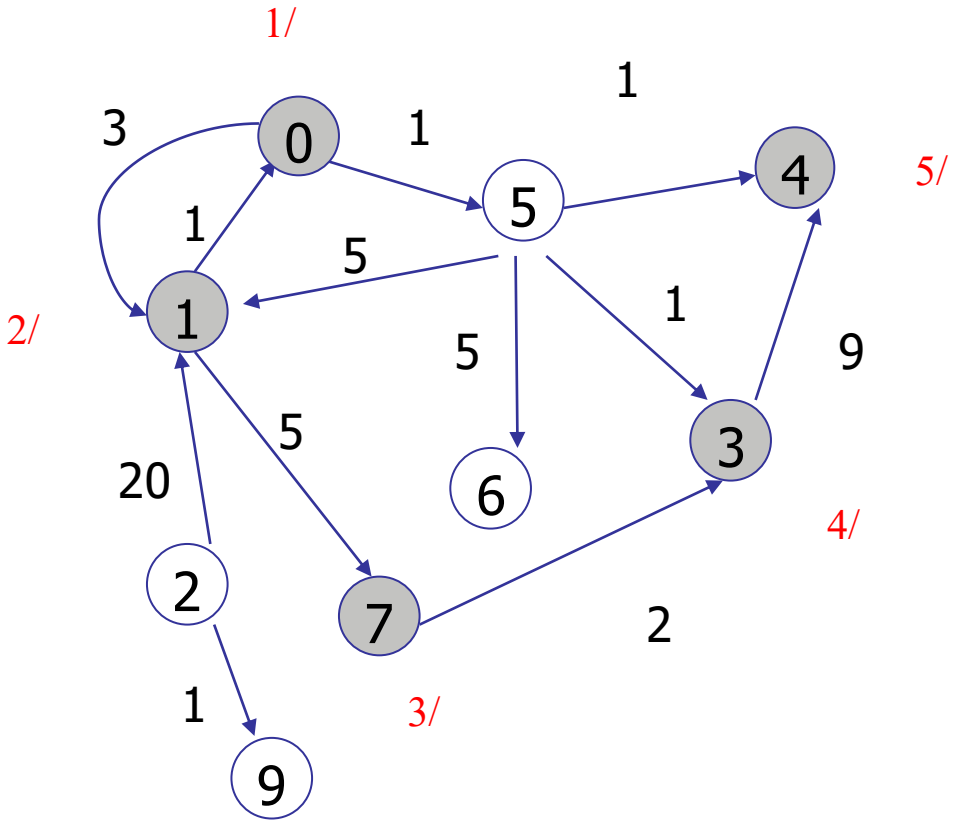
Caminhamento em profundidade



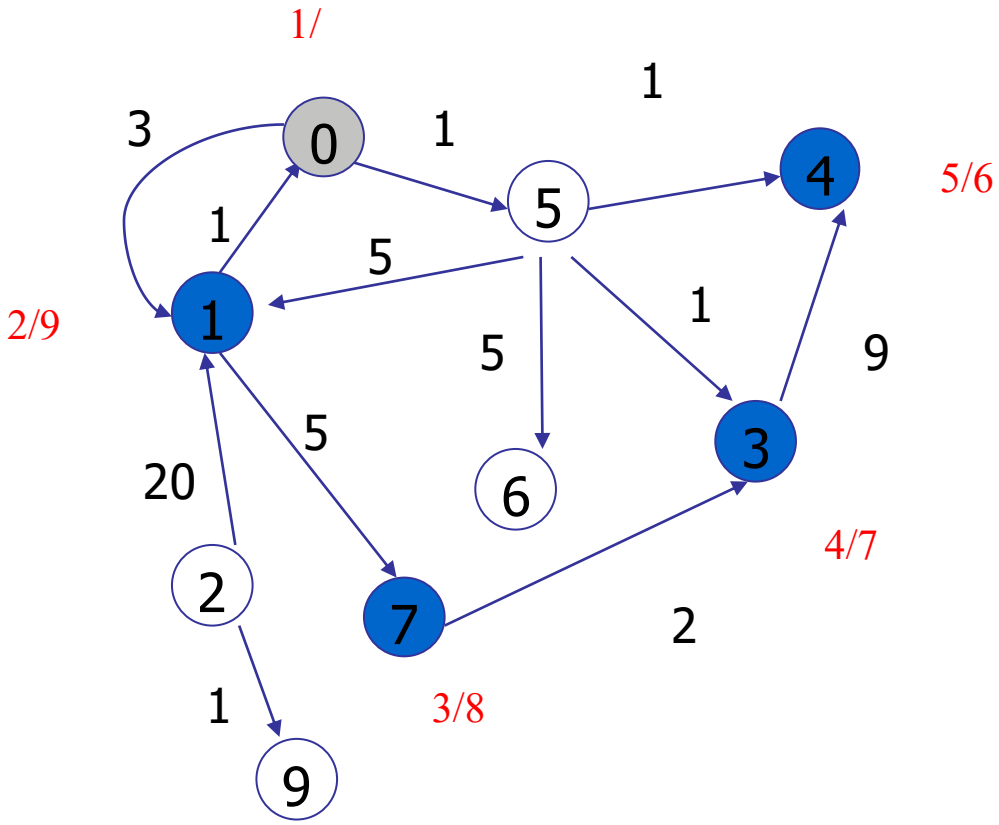
Caminhamento em profundidade



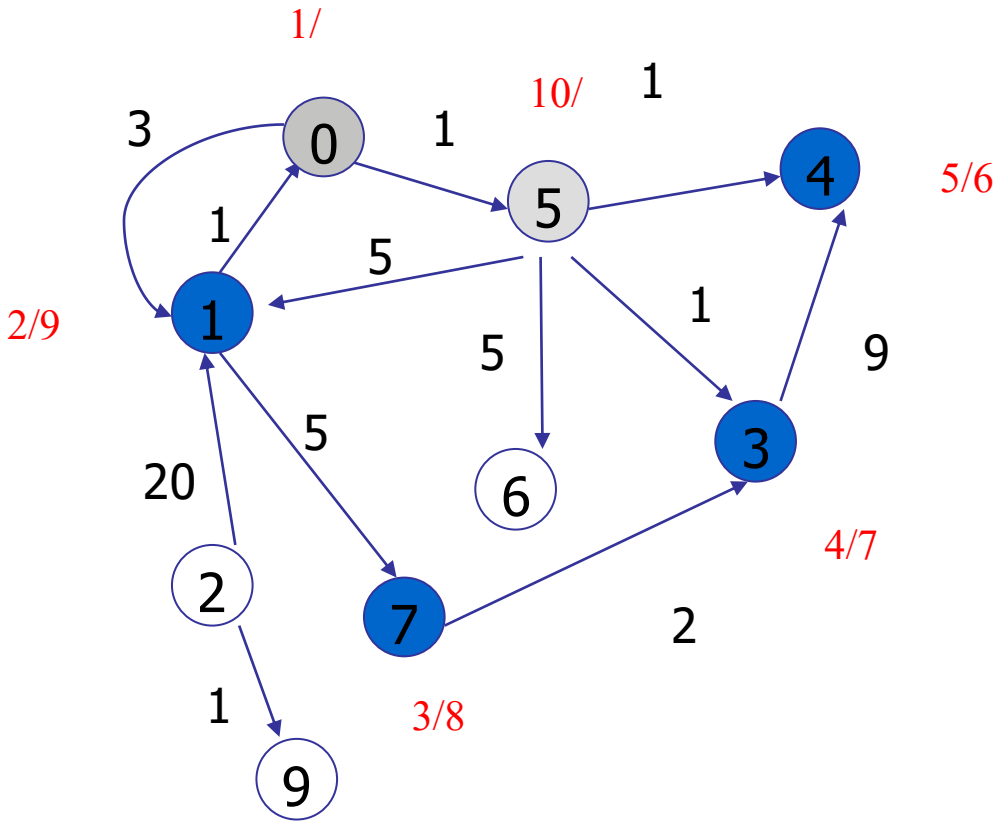
Caminhamento em profundidade



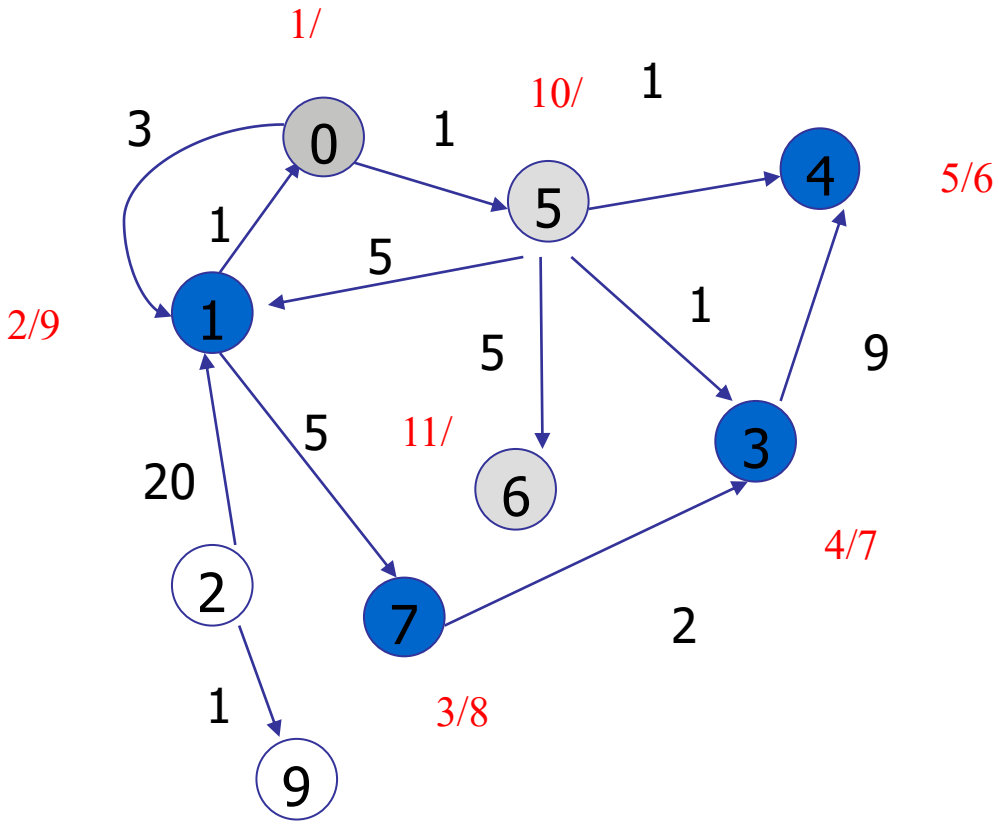
Caminhamento em profundidade



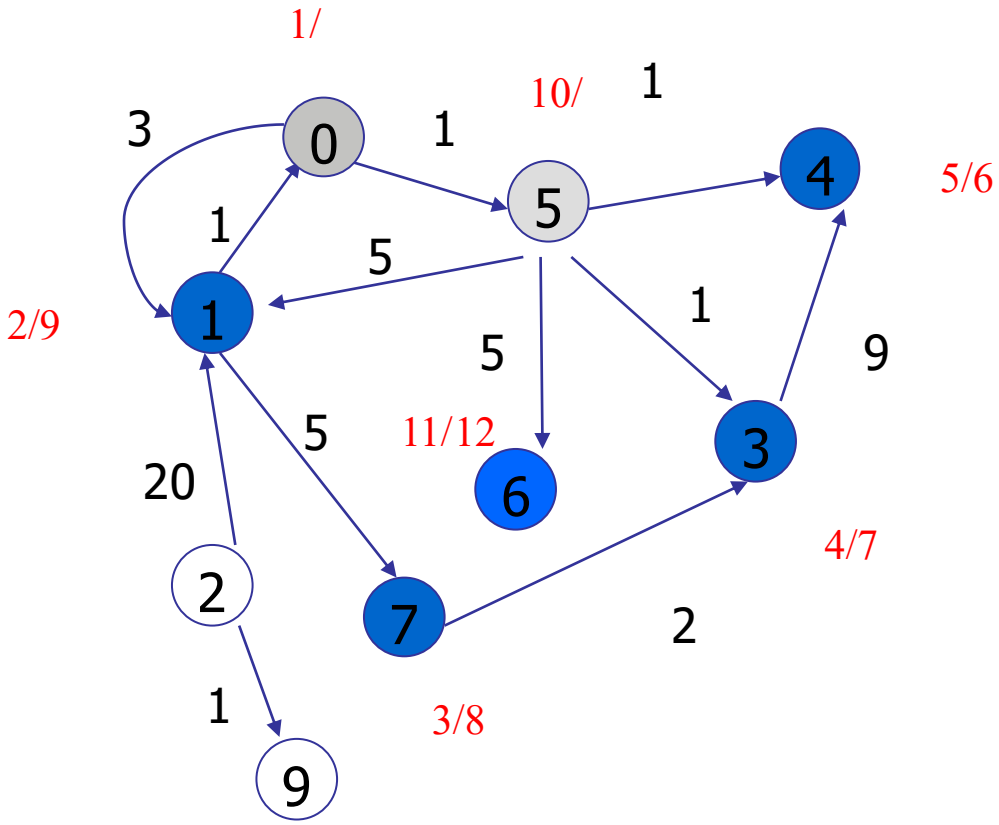
Caminhamento em profundidade



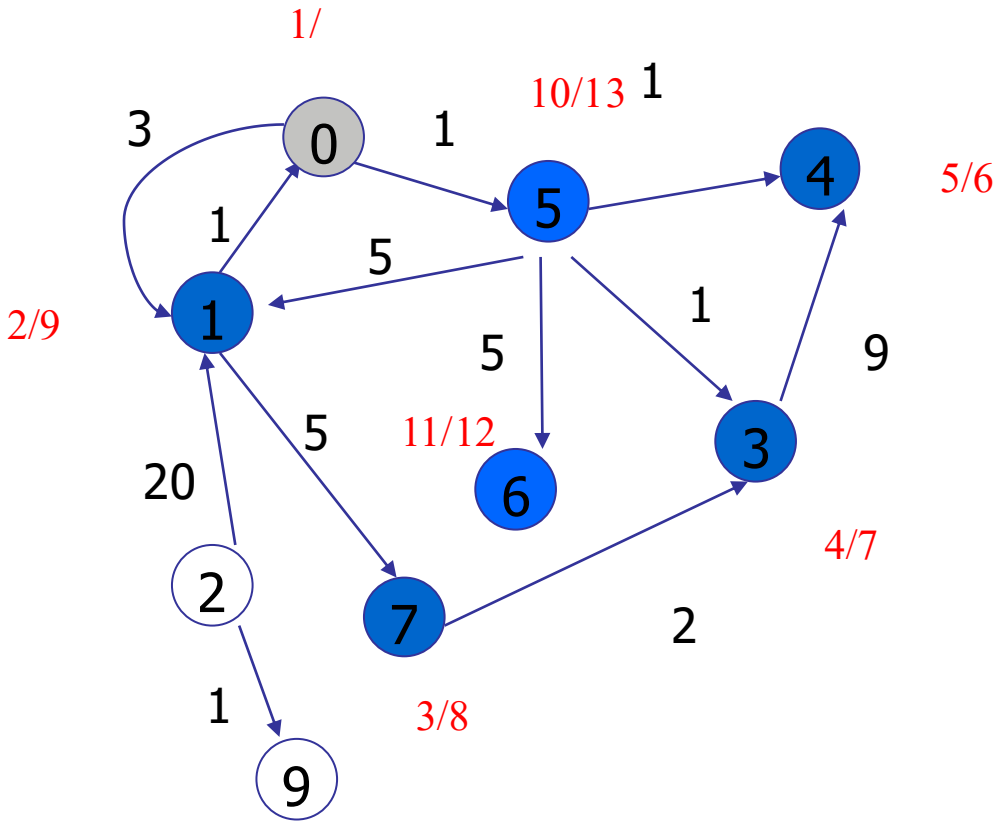
Caminhamento em profundidade



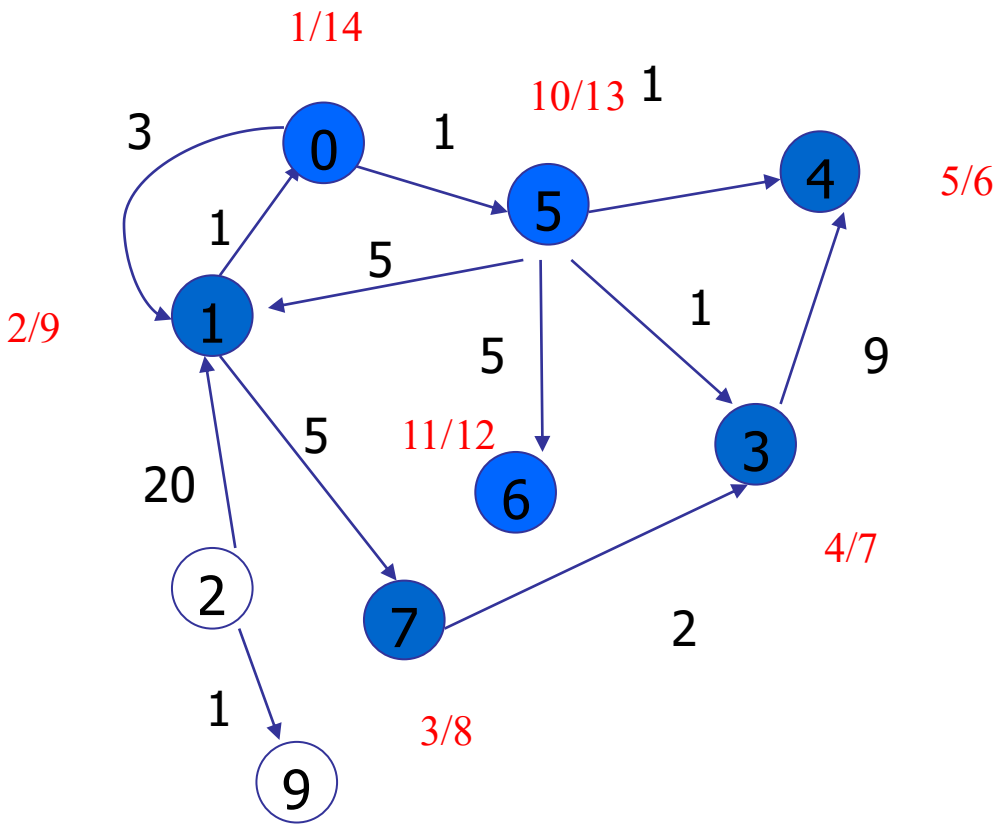
Caminhamento em profundidade



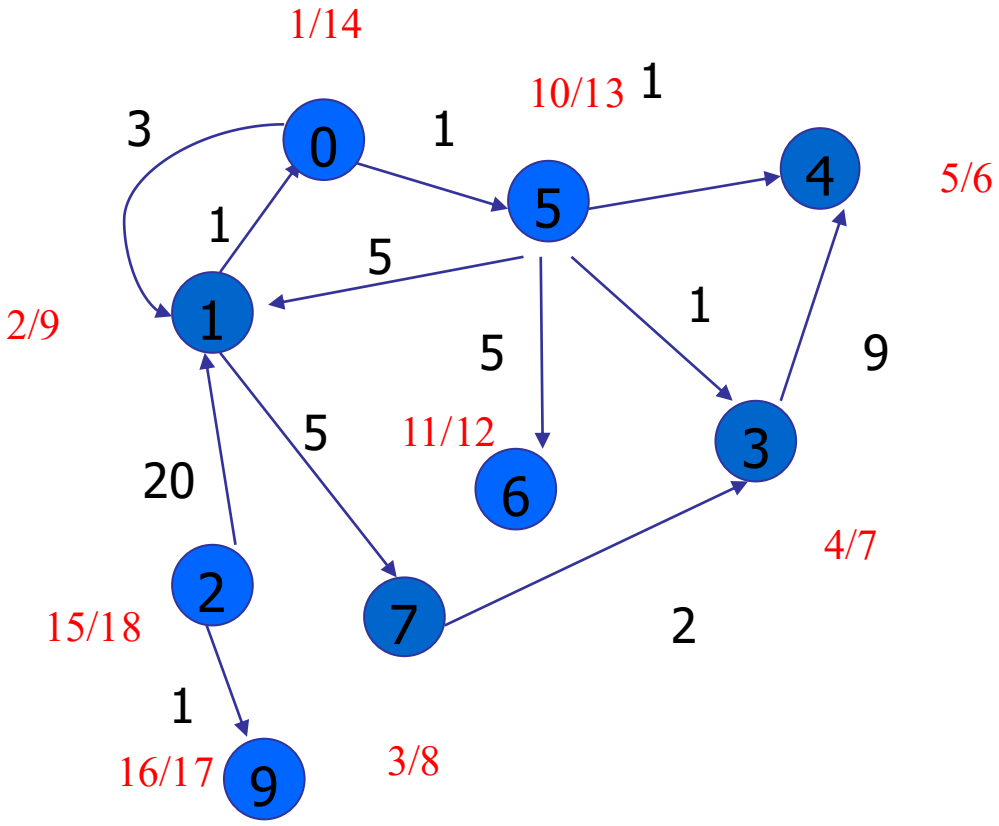
Caminhamento em profundidade

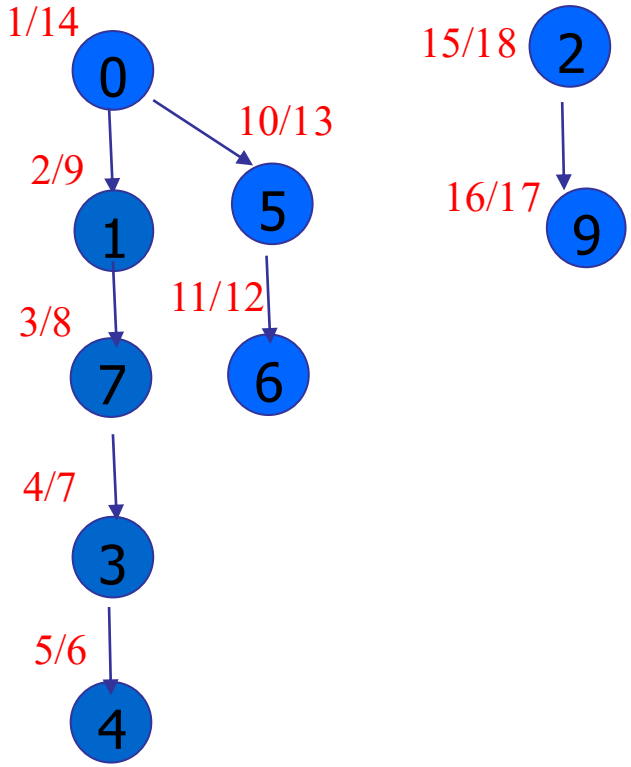


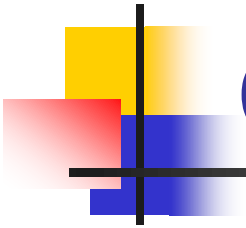
Caminhamento em profundidade



Caminhamento em profundidade



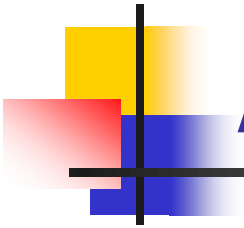




Caminhamento em profundidade

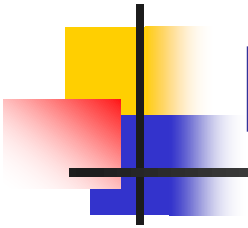
- Propriedades:
  - Intervalos entre início e término são disjuntos para vértices de árvores distintas
  - Intervalos de descendentes de um vértice estão contidos nos intervalos de seus ancestrais

Ainda sobre intervalos...



- Para todo vértice $d[u] < f[u]$
- Um vértice u *é*
  - Branco antes de $d[u]$
  - Cinza entre $d[u]$ e $f[u]$
  - Preto (azul nos slides anteriores) dai em diante

Caminhamento em Profundidade

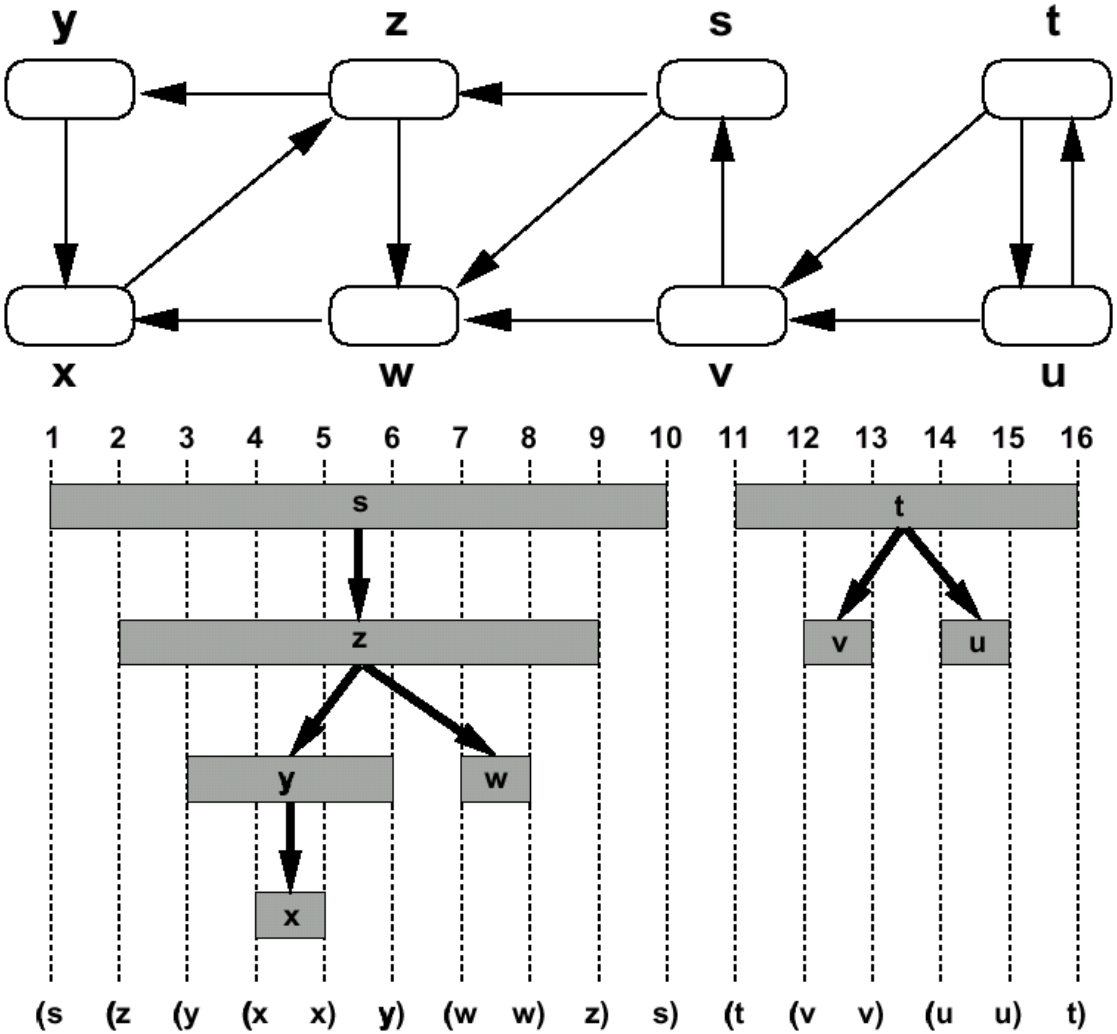


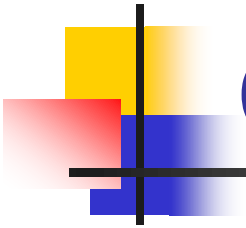
- Gera uma floresta de árvores de caminhamento

Teorema dos Parentesis

- Tempos de descoberta e fim podem ser associados a uma estrutura de parentesis
  - A descoberta de *u* é representada com "(u"
  - O fim do processmento de *u* é representado com "u)"
  - A história das descobertas e fins gera uma expressão bem formada (parentesis corretamente aninhados)
- Intuição para prova:
  - Quaisquer dois intervalos são disjuntos ou inclusos
  - Intervalos sobrepostos implicaria no término do processamento de ancestral antes que seus descendentes terminassem de ser processados ou no início do processamento de um descendente antes do ancestral

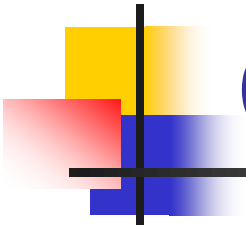
Teorema dos Parentesis (2)





Caminhamento em profundidade

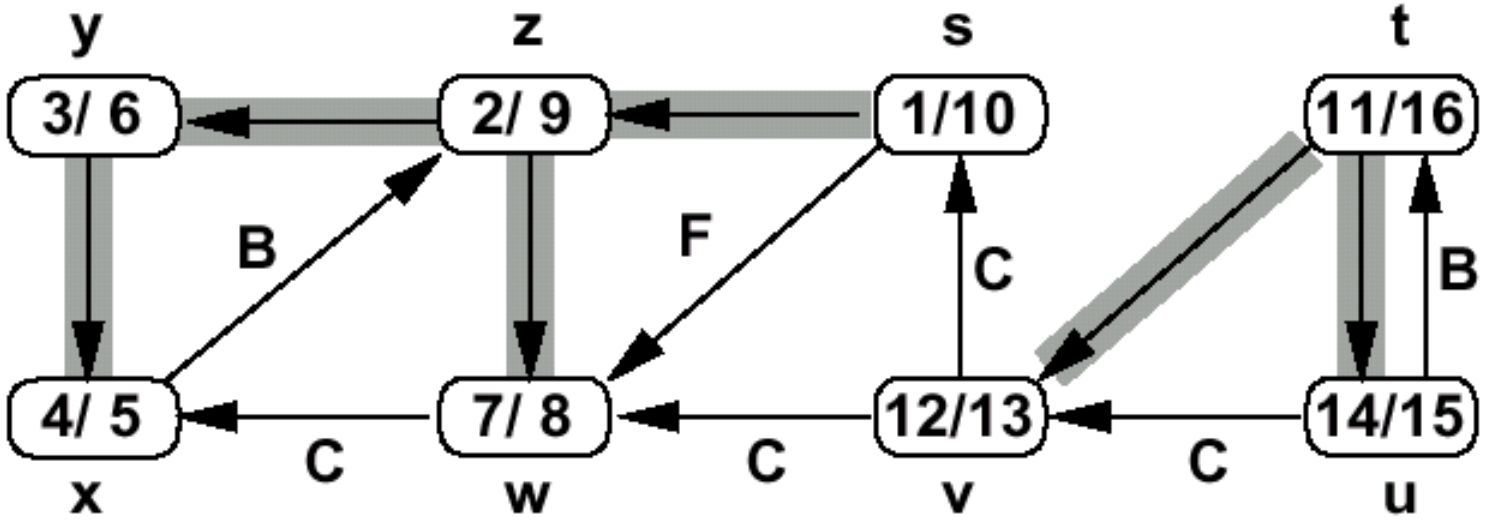
- Algoritmo classifica arestas em quatro categorias:
  - Arestas de árvore: são usadas na floresta primeiro em profundidade
  - Arestas reversas (ou de retorno): conectam um vértice a um ancestral
  - Arestas diretas: conectam um vértice a um descendente
  - Arestas cruzadas: ligam um vértice a um outro que não pertence à mesma árvore na floresta de caminhamento



Classificação de Arestas

- Arestas de árvore (cinza para branco)
  - Indicam a descoberta de novos vértices
- Arestas reversas (cinza para cinza)
  - Do descendente para o ancestral
- Arestas diretas (cinza para preto)
  - Do ancestral para descendentes
- Areas de cruzamento (cinza para preto)
  - Entre árvores ou sub árvores (restantes)

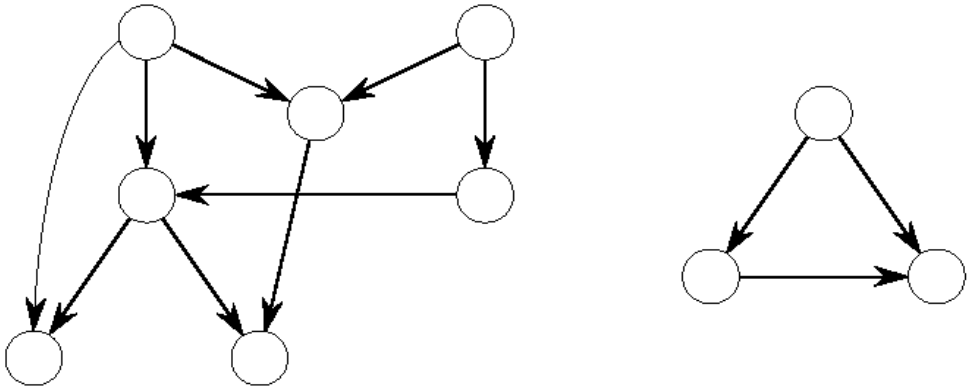
Classificação de Arestas



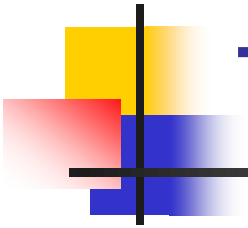
B – Reversas
C – Cruzamento
F – Diretas

DAGs

- Grafos Dirigidos Acíclicos ou Direct Aciclic Graphs (DAGS)



- Usados geralmente para indicar a precedência entre eventos
- Uma ordenação total para os eventos pode ser gerada usando o algoritmo de **Ordenação Topológica**



Teorema do DAG

- Seja *G = (V,E)* um grafo dirigido
- *G* é acíclico sse uma DFS (busca em profundidade) sobre ele não gera arestas reversas

Teorema do DAG (2)

- Prova – parte a
  - **Suponha que existe uma aresta reversa (*u*,*v*);**
  - Então, *v* é um ancestral de *u* na floresta DFS
  - Portanto, existe um caminho de v até u em G e (*u*,*v*) fecha o ciclo

Teorema do DAG (2)

- Prova – parte b
  - **Suponha que existe um ciclo *c***
  - Seja *v* o primeiro vértice no ciclo *c* a ser descoberto e *u* o predecessor de v no ciclo
  - Quando *v* for descoberto, o ciclo de v até u é todo branco
  - É necessário visitar todos os nodos alcançáveis neste caminho antes de retornar de DFS-Visit(*v*), ou seja até que *u* seja um descente de *v*
  - Então, no DFS (*u,v*) é uma aresta reversa

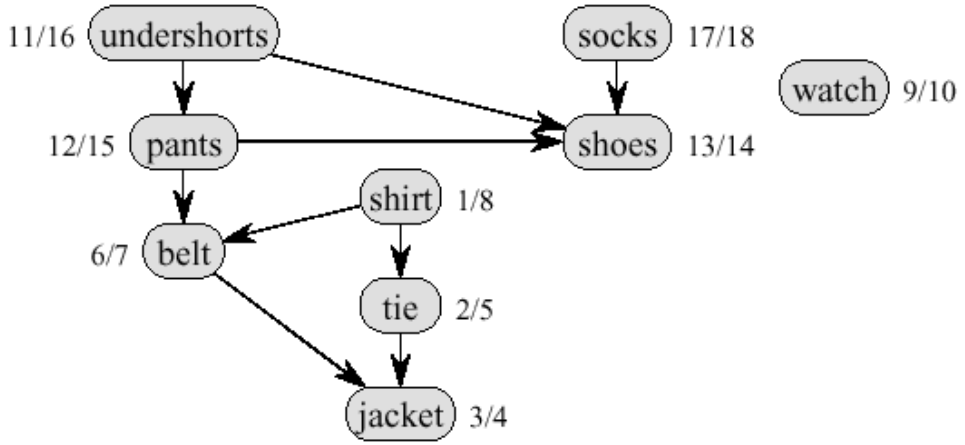
Ordenação Topológica

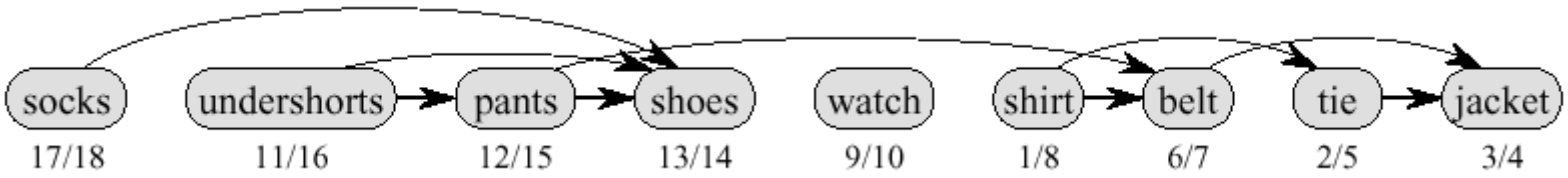
- Ordenação de um DAG
- Ordenaçao linear de todos os vértices tal que, para toda aresta (u,v) no DAG, u aparece antes de v na ordenação

TopSort(G)
1) Execute DFS(G) para computar os tempos de término $f[v]$ de cada vértice v
2) Gere uma lista ordenada de acordo com $f[v]$ *(em ordem decrescente)*

TopSort – Exemplo

Relações de Precedência

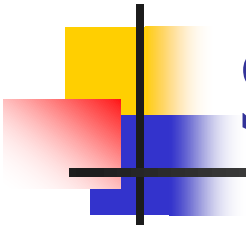




Componentes fortemente conectados

- Dados G=(V,E):

- Um conjunto máximo de vértices C contido em V tal que exista um caminho de u até v e de v até u para qualquer par de vértices pertencentes a C.



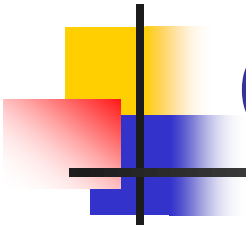
Solução para o problema

1. Chamar DFS(G) para calcular o tempo de término f[u] para cada vértice
2. Calcular G^t
3. Chamar DFS(G^t), mas considerar vértices em ordem decrescente de f[u](calculada na linha 1) no loop principal
4. Dar saída aos vértices de cada árvore na floresta primeiro na profundidade formada na linha 3 como um componente fortemente conectado separado

Por que funciona ?

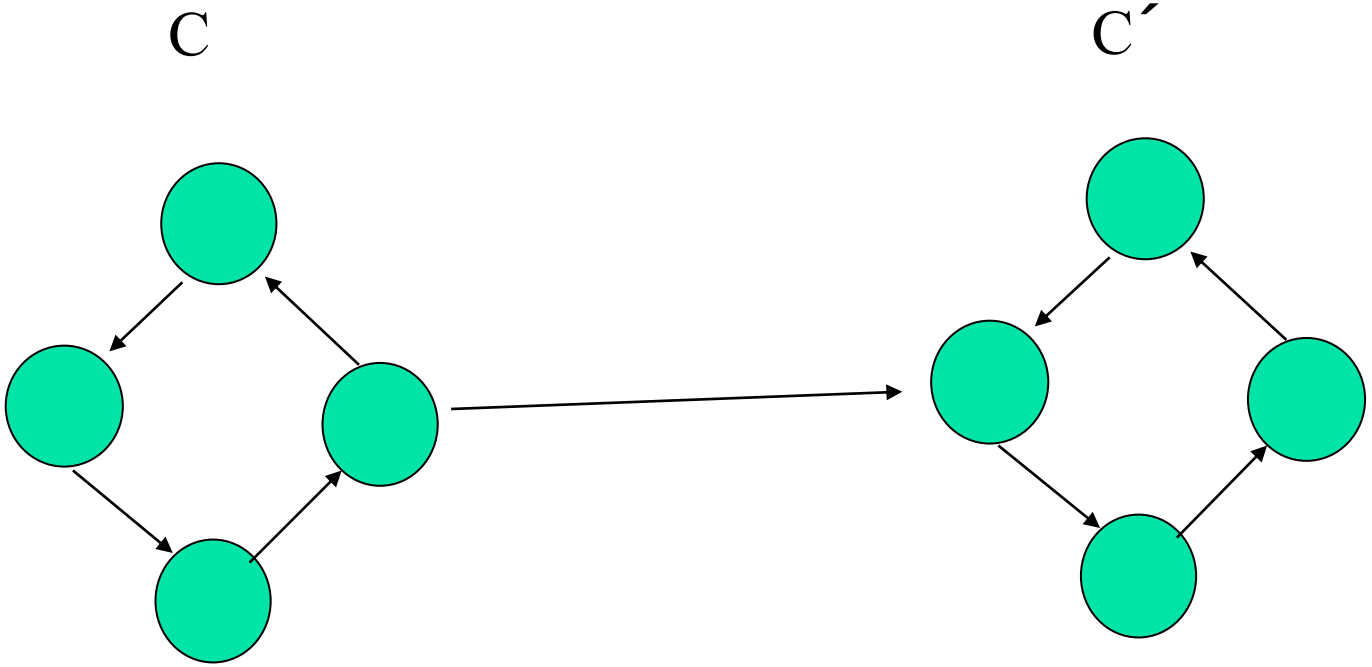
1 - Sejam C e C´ CFCs distintos em G=(V,E), seja u,v $\in$ C, seja u´ e v´ $\in$ C´ e suponha que existe u $\rightarrow$ u´ então não há v´ $\rightarrow$v

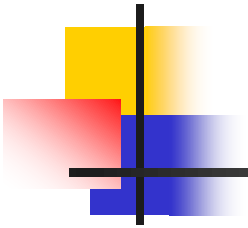
- Percebe-se mostrando que os dois componentes poderiam ser unidos se isso ocorresse



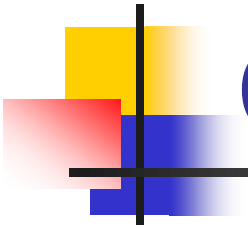
CFCs

- Considere f(C) o maior termino e d(C) a menor descoberta
- 2 – Suponha que exista aresta (u,v) $\in$ E, onde u $\in$ C e v $\in$ C´. Então f(C) > f(C´)
- Há dois casos, d(C) < d(C´) ou
- d(C) >d(C´)


C
C´



- 3 -Suponha que exista aresta $(u,v) \in E^t$, onde $u \in C$ e $v \in C´$. Então $f(C) < f(C´)$.
- Usamos o resultado anterior para concluir 3



Conclusão

- A segunda pesquisa em G^t começa com um componente de tempo de término máximo
- A pesquisa é iniciada em um componente conectado e não há aresta deste componente para outro. O caminhamento atinge todos os vértices do primeiro CFC
- O próximo elemento a ser visitado começa um segundo CFC e o mesmo raciocínio se aplica